

Chapter 7: Congestion Control and Quality of Services

Some of the Causes of Congestion

- Too many host in broadcast domain
- Low bandwidth
- Outdated hardware
- Bad configuration management
- Poor network design

Congestion Control

Open Loop Congestion Control

In open-loop congestion control, policies are applied to prevent congestion before it happens. In these mechanisms, congestion control is handled by either the source or the destination

Closed-Loop Congestion Control

Closed-loop congestion control mechanisms try to alleviate congestion after it happens.

Open Loop Congestion Control

Retransmission Policy

If a sent packet is lost or corrupted, the packet needs to be retransmitted. Retransmission in general may increase congestion in the network. However, a good retransmission policy can prevent congestion.

Window Policy

The Selective Repeat window is better than the Go-Back-N window for congestion control. In the Go-Back-N window, sends from last lost frame that may cause duplication and congestion. The Selective Repeat window, on the other hand, tries to send the specific packets that have been lost or corrupted.

Acknowledgment Policy

If the receiver does not acknowledge every packet it receives, it may slow down the sender and help prevent congestion. Several approaches are used in this case. A receiver may send an acknowledgment only if it has a packet to be sent or a special timer expires. A receiver may decide to acknowledge only N packets at a time. Sending fewer acknowledgments means imposing less load on the network.

Discarding Policy

A good discarding policy by the routers may prevent congestion. For example, in audio transmission, if the policy is to discard less sensitive packets when congestion is likely to happen, the quality of sound is still preserved and congestion is prevented or alleviated.

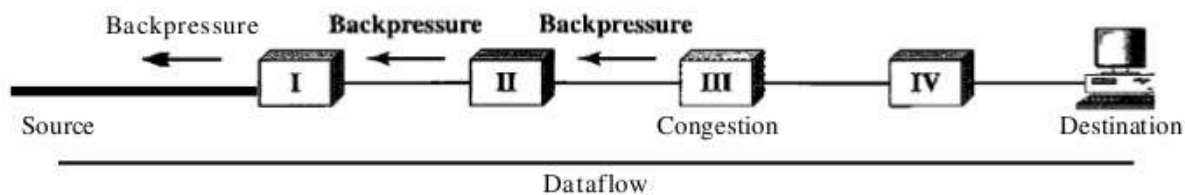
Admission Policy

Switches in a flow first check the resource requirement of a flow before admitting it to the network. A router can deny establishing a virtual- circuit connection if there is congestion in the network or if there is a possibility of future congestion.

Closed-Loop Congestion Control

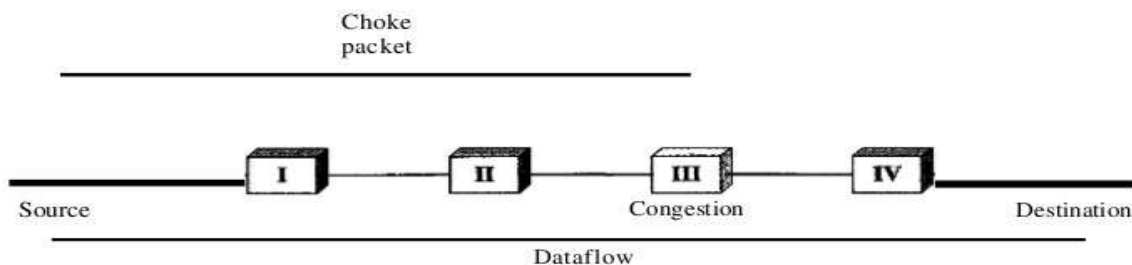
Back-pressure

The technique of backpressure refers to a congestion control mechanism in which a congested node stops receiving data from the immediate upstream node or nodes. This may cause the upstream node or nodes to become congested, and they, in turn, reject data from their upstream nodes or nodes and so on.



Choke Packet

A choke packet is a packet sent by a node to the source to inform it of congestion. Difference from backpressure - the warning is from one node to its upstream node, although the warning may eventually reach the source station. In the choke packet method, the warning is from the router, which has encountered congestion, to the source station directly. The intermediate nodes through which the packet has traveled are not warned.



Implicit Signaling

In implicit signaling, there is no communication between the congested node or nodes and the source. The source guesses that there is a congestion somewhere in the network from other symptoms. For example, when a source sends several packets and there is no acknowledgment for a while, one assumption is that the network is congested. The delay in receiving an acknowledgment is interpreted as congestion in the network; the source should slow down.

Explicit Signaling

In the choke packet method, a separate packet is used ; in the explicit signaling method, the signal is included in the packets that carry data.

Backward Signaling

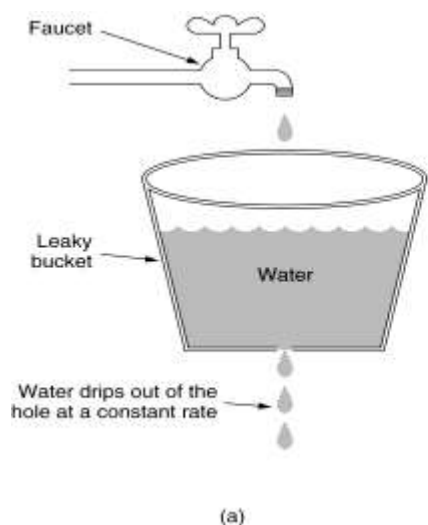
A bit can be set in a packet moving in the direction opposite to the congestion. This bit can warn the source that there is congestion and that it needs to slow down to avoid the discarding of packets.

Forward Signaling

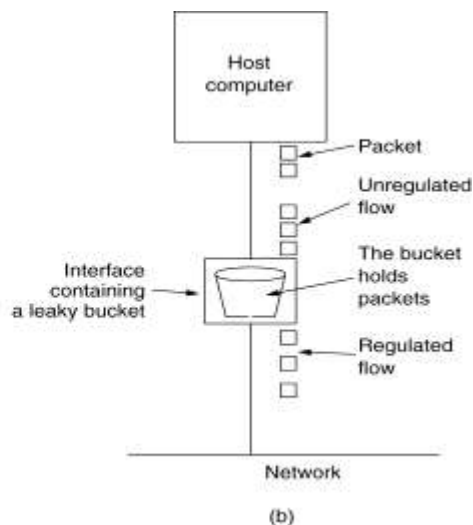
A bit can be set in a packet moving in the direction of the congestion. This bit can warn the destination that there is congestion. The receiver in this case can use policies, such as slowing down the acknowledgments, to alleviate the congestion.

Traffic Shaping Algorithm

The Leaky Bucket Algorithm



(a) A leaky bucket with water

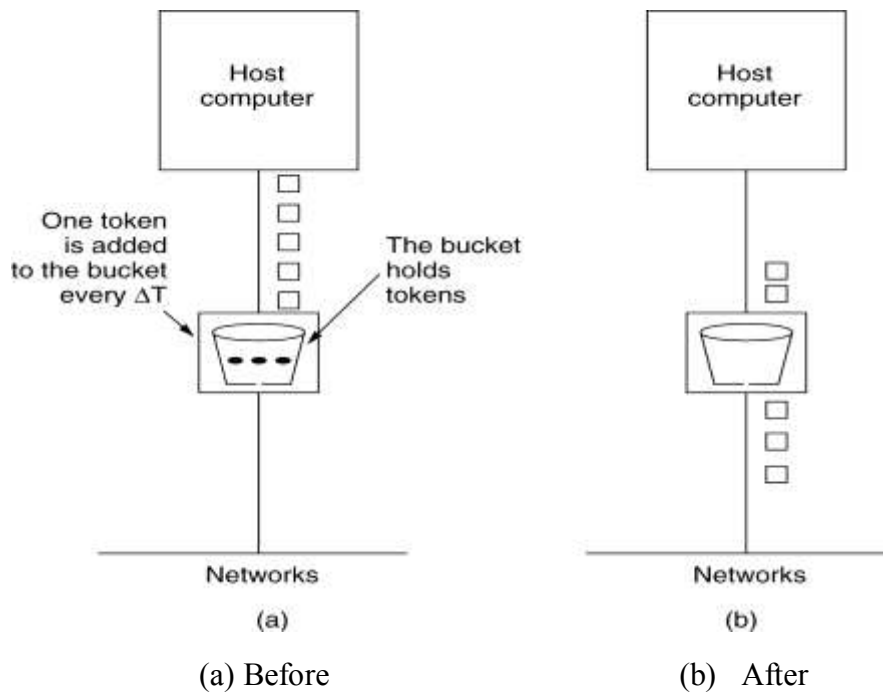


(b) a leaky bucket with packets.

Try to imagine a bucket with a small hole in the bottom, as illustrated in figure. No matter the rate at which water enters the bucket, the outflow is at a constant rate, R , when there is any water in the bucket and zero when the bucket is empty. Also, once the bucket is full to capacity, any additional water entering it spills over the sides and is lost. This bucket can be used to shape or police packets entering the network.

Conceptually, each host is connected to the network by an interface containing a leaky bucket. If a packet arrives when the bucket is full, the packet must either be queued until enough water leaks out to hold it or be discarded.

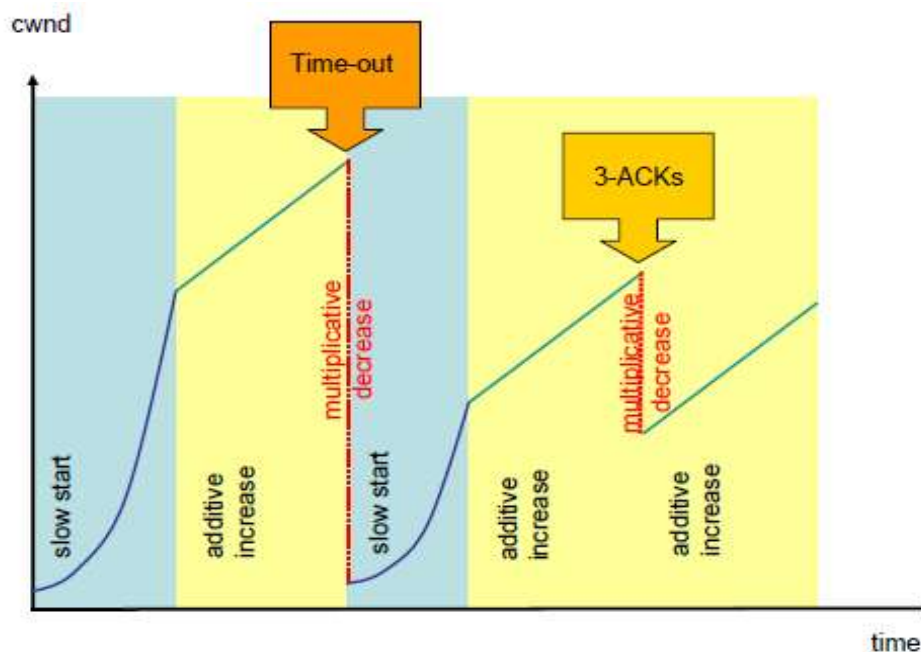
The Token Bucket Algorithm



A different but equivalent formulation is to imagine the network interface as a bucket that is being filled, as shown in figure. Now, to send a packet we must be able to take water, or tokens, as the contents are commonly called, out of the bucket (rather than putting water into the bucket). No more than a fixed number of tokens, can accumulate in the bucket. If the bucket is empty, we must wait until more tokens arrive before we can send another packet.

TCP congestion control: additive increase, multiplicative decrease (AIMD)

- Approach: increase transmission rate (window size), probing for usable bandwidth, until loss occurs.
 - additive increase: increase CongWin by 1 MSS every RTT until loss detected
 - multiplicative decrease: cut CongWin in half after loss
- It starts with slow start phase that exponentially increase CWND and continues to congestion avoidance threshold.
- When it reaches the congestion avoidance threshold then it additively increases CWND
- It follows multiplicative decreasing rate upon congestion detection.



TCP congestion control: TCP Slow Start

- A sender starts transmission with a slow rate
- Increases the rate as long as its rate is below a threshold.
- When the threshold is reached, the rate is decreased to avoid congestion.
- When connection begins, increase rate exponentially until first loss event:
 - double CongWin every RTT
 - done by incrementing CongWin **for** every ACK received

